Olympia Track

Sports Management System

UML Diagrams | ER Diagram | UML-to-ER Conversion

Course: CS 432: Databases
Assignment: Assignment 1 (Track 1)
Semester: II (2025–2026)
Institute: Indian Institute of Technology, Gandhinagar
Github: https://github.com/Venkat-2341/Database-Assignment-1
Date: February 15, 2026

Contents

1. System Overview

1.1 Problem Statement

Many schools, colleges, and sports clubs rely on manual registers, spreadsheets, or disconnected tools to manage their sports activities. Information about event schedules, team members, equipment, and player performance is stored in different places, leading to scheduling clashes, unreturned equipment, and incorrect participation records. Coaches struggle to track player improvement over time. As sports programs expand, manual management becomes disorganized, resulting in wasted time and missed opportunities for athlete development.

1.2 Proposed Solution

Olympia Track is a comprehensive Sports Management System built on a well-structured relational database. It provides centralized management for:

- **Tournaments** — Organizing high-level championships that contain multiple events.
- **Members** — Players, coaches, and administrators with role-based profiles.
- **Teams** — Formation, coach assignment, captain selection, and player roster management.
- **Events** — Scheduling, venue assignment, and clash prevention.
- **Equipment** — Inventory tracking, issue/return management.
- **Performance** — Metric logging and improvement analysis over time.

1.3 System Statistics

Metric	Required	Delivered
Distinct Entities	≥ 5	**9**
Total Tables	≥ 10	**13**
Core Functionalities	≥ 5	**6**
Rows per Table	10–20	**15–20**
Foreign Key Relationships	–	**14**
CHECK Constraints	–	**8**
NOT NULL Columns per Table	≥ 3	**3–8**

Table 1: Assignment compliance summary

1.4 Entity Summary

#	Table	Type	Purpose	Primary Key
1	Tournament	Strong Entity	Grouping events into championships	TournamentID
2	Member	Strong Entity	All users: players, coaches, admins	MemberID
3	Sport	Strong Entity	Types of sports offered	SportID
4	Team	Strong Entity	Sports teams (Group or Individual)	TeamID
5	TeamMember	Junction	Players in teams with captain status	(TeamID, MemberID)
6	Venue	Strong Entity	Sports facilities	VenueID
7	Event	Strong Entity	Competitions/events	EventID
8	Participation	Junction	Teams in events	ParticipationID
9	Equipment	Strong Entity	Sports gear inventory	EquipmentID
10	EquipmentIssue	Tracking	Equipment issue/return	IssueID
11	PracticeSession	Strong Entity	Team practices	SessionID
12	PerformanceLog	Strong Entity	Player metrics over time	LogID
13	MedicalRecord	Strong Entity	Injury/health records	RecordID

Table 2: Complete list of 13 tables

2. UML

UML (Unified Modeling Language) diagrams are shown below to provide a high-level view of the system.

2.1 Tournament

A tournament (e.g., "Hallabol, IITGN") is composed of multiple events.

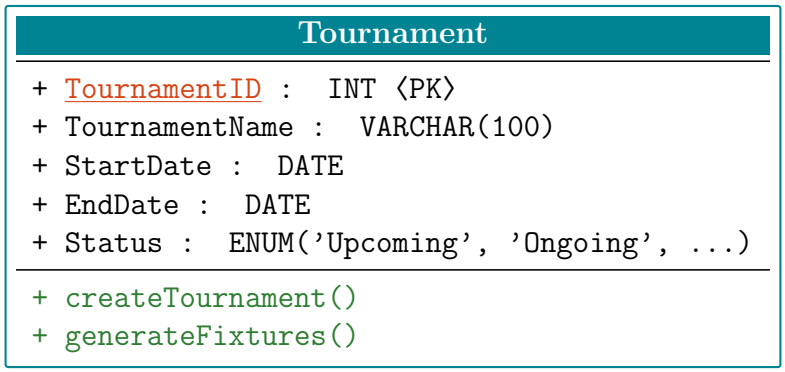

Figure 1: UML Class: Tournament

2.2 Member

The central entity representing all system users.

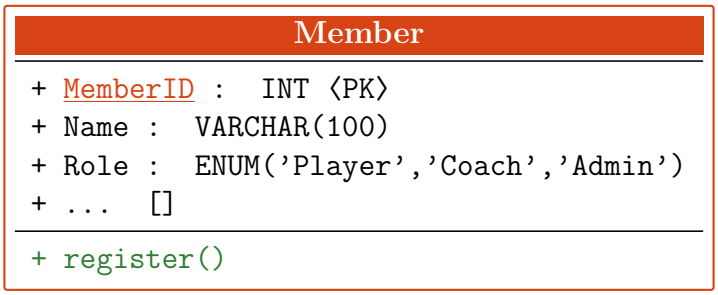

Figure 2: UML Class: Member

2.3 Team

Supports both "Group Teams" (Football) and "Teams of One" (Tennis). Includes a Captain link.

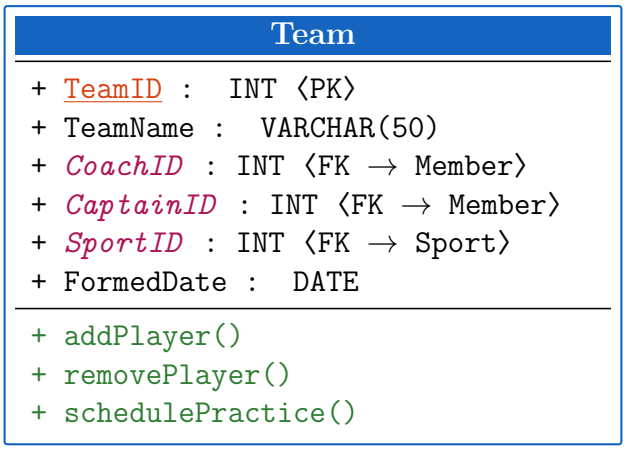

Figure 3: UML Class: Team

2.4 Event

Linked to a parent Tournament.

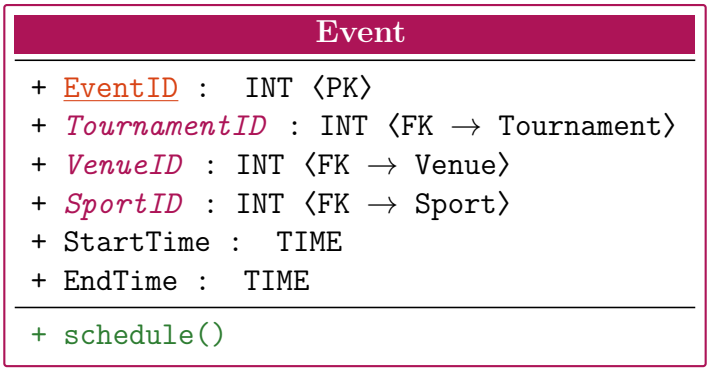

Figure 4: UML Class: Event

2.5 Equipment

Renamed `Condition` to `EquipmentCondition` to avoid SQL keywords.

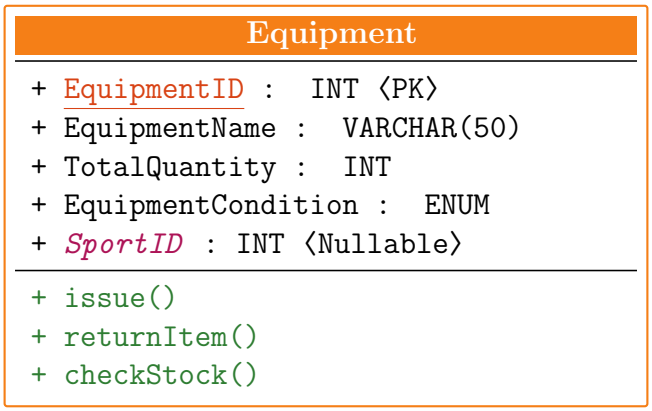

Figure 5: UML Class: Equipment

2.6 MedicalRecord

Renamed `Condition` to `MedicalCondition`.

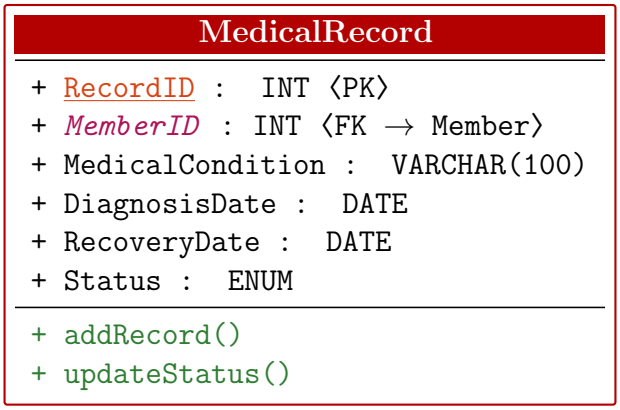

Figure 6: UML Class: MedicalRecord

2.7 Sport

Defines the types of sports offered. `Category` distinguishes Individual, Team, and Dual sports.

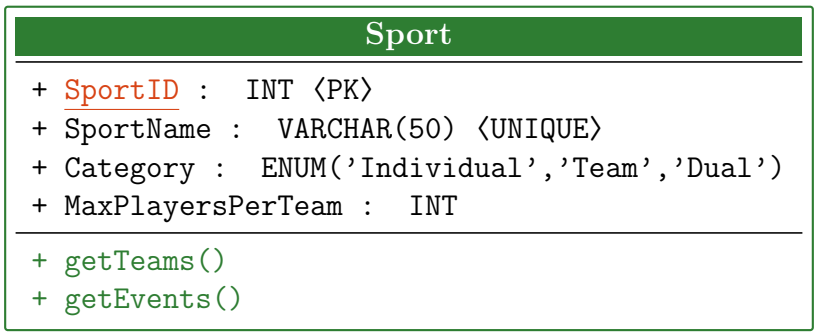

Figure 7: UML Class: Sport

2.8 Venue

Stores information about sports facilities. Events and practice sessions reference venues to ensure proper scheduling.

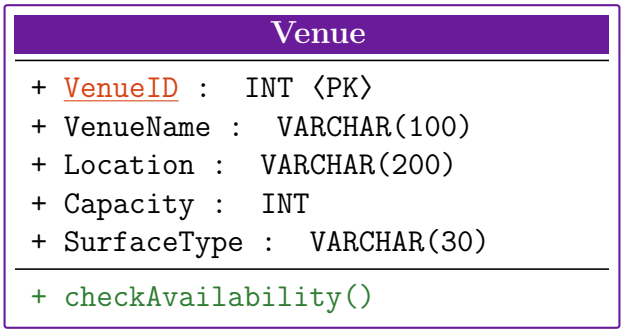

Figure 8: UML Class: Venue

2.9 PerformanceLog

Tracks measurable performance metrics (e.g., 100m time, goals scored) per player over time, enabling coaches to analyze improvement trends.

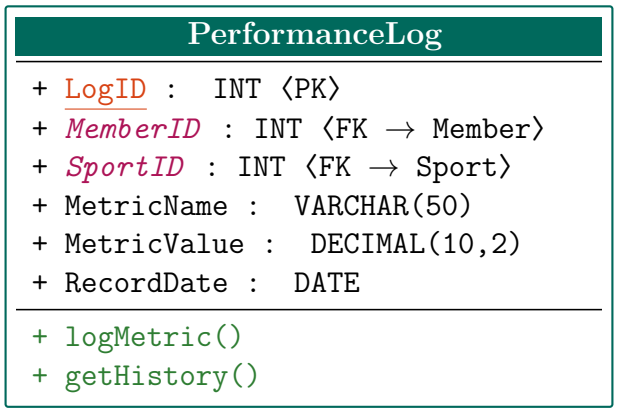

Figure 9: UML Class: PerformanceLog

2.10 PracticeSession

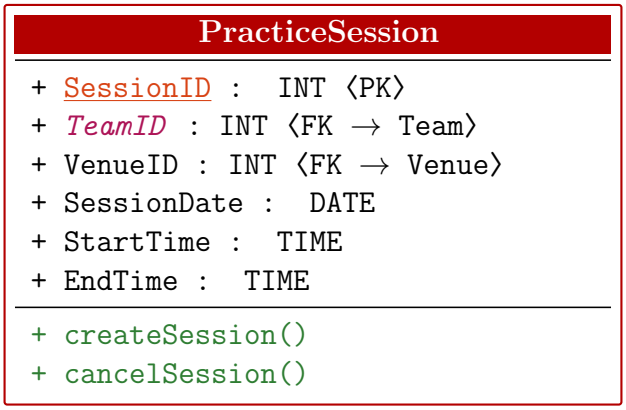

Figure 10: UML Class: PracticeSession

3. UML Relationships

From	To	Mult.	Type	Description
Tournament	Event	1:0..*	Composition	A Tournament consists of multiple Events.
Event	Participation	1:0..*	Association	Events track results via the Participation entity.
Member	Participation	1:0..*	Association	Members (Players) record scores in Events.
Team	TeamMember	1:0..*	Aggregation	A Team is composed of various Team Members.
Member	TeamMember	1:0..*	Aggregation	A Member can be part of multiple Teams over time.
Team	PracticeSession	1:0..*	Composition	Teams own and schedule their specific Practice Sessions.
Member	MedicalRecord	1:0..*	Composition	Each Medical Record belongs strictly to one Member.
Member	EquipmentIssue	1:0..*	Association	Tracks which Member borrowed specific equipment.
Equipment	EquipmentIssue	1:0..*	Composition	Equipment stock tracks individual issue instances.
Event	Venue	0..*:1	Association	Multiple Events can be held at a single Venue.
Sport	Team	1:0..*	Association	A Sport categorizes many different Teams.

Table 3: Updated UML Relationship Summary based on Class Diagram

3.1 UML Relationship Diagram

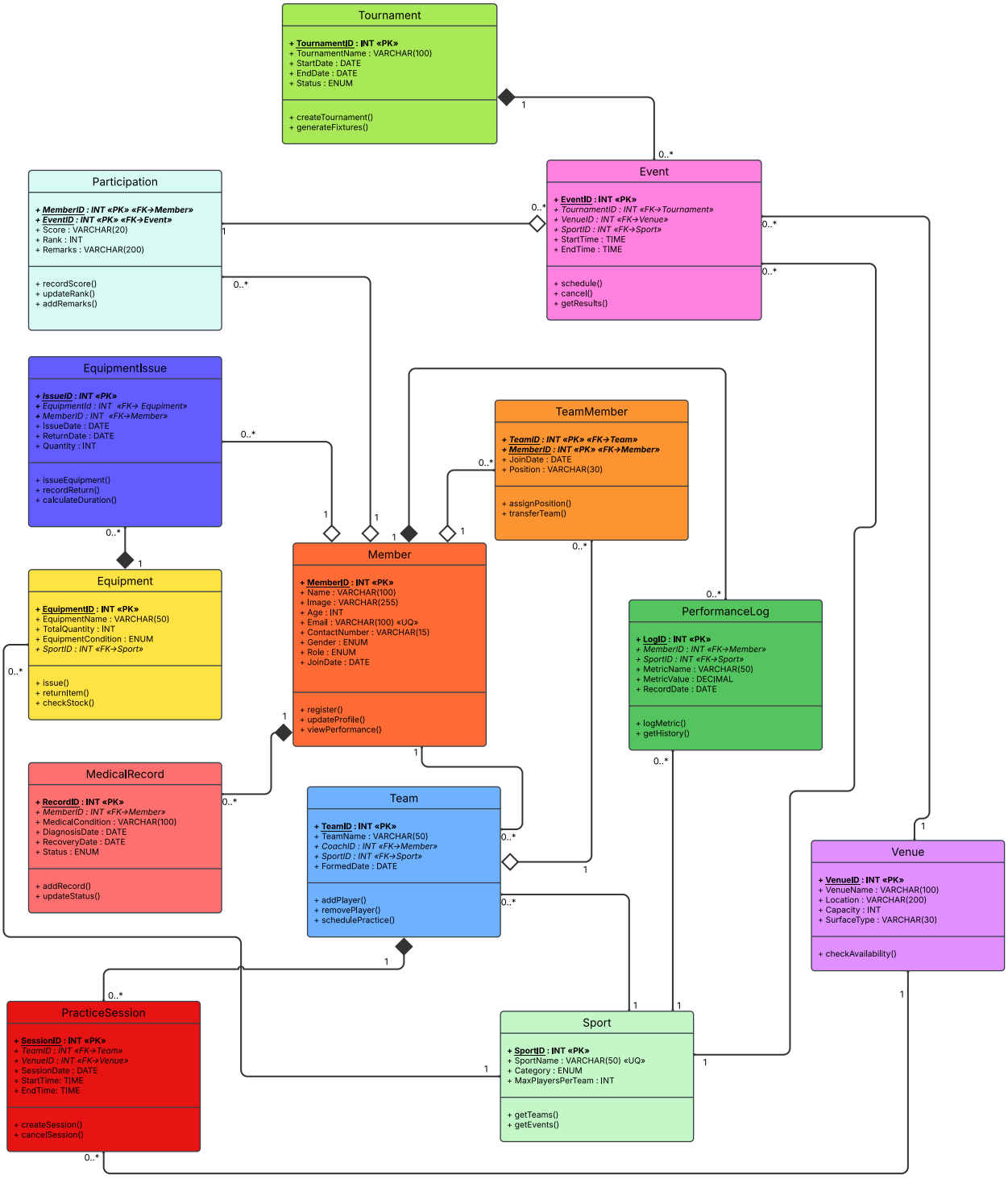

Figure 11: UML relationship diagram

4. ER Diagram

4.1 ER Entity-Relationship Map

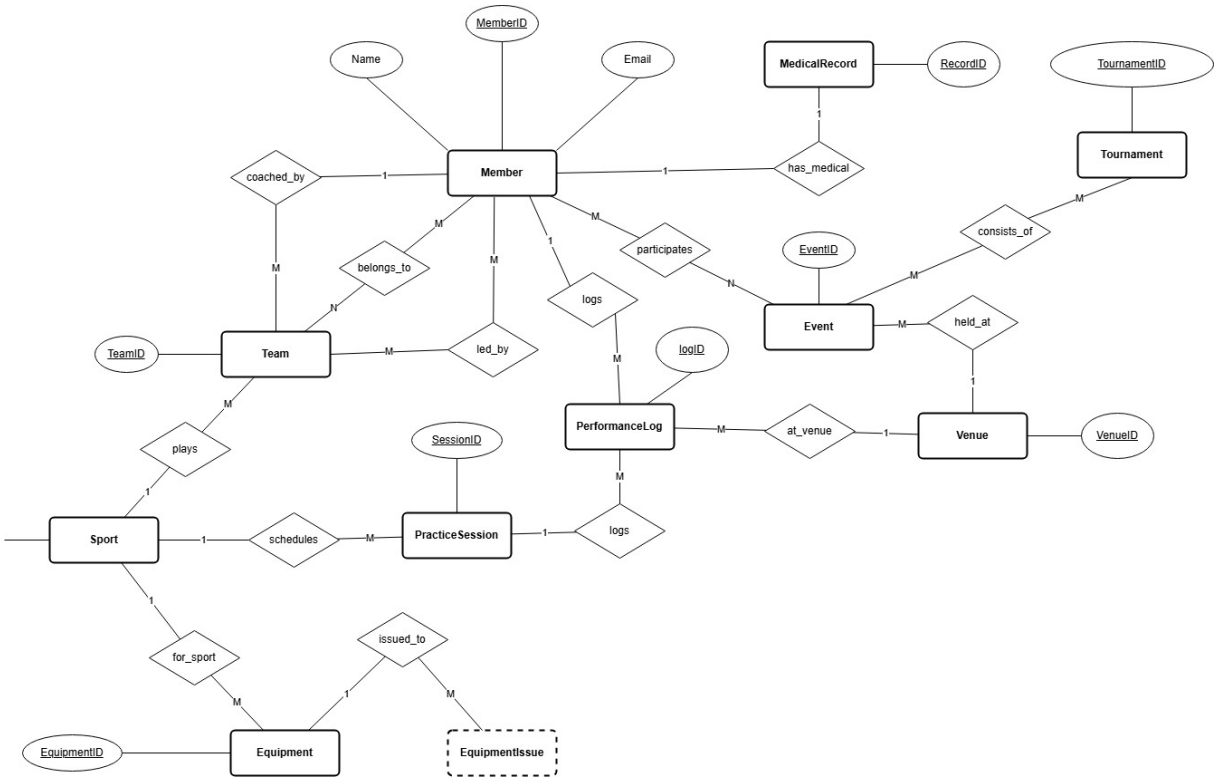

Figure 12: Entity relationship diagram

4.2 ER Attributes per Entity

Entity	Attributes
Tournament	**TournamentID**, NameNN, StartDateNN, EndDateNN, Status
Member	**MemberID**, NameNN, Image, AgeNN, EmailNNUQ, RoleNN
Team	**TeamID**, NameNN, *CoachID*, *CaptainID*, *SportID*NN
TeamMember	***TeamID***, ***MemberID***, JoinDateNN, Position, IsCaptain
Event	**EventID**, NameNN, *TournamentID*, *VenueID*, *SportID*, StartTime, EndTime
Participation	**ParticipationID**, *TeamID*NN, *EventID*NN, Score, EventRank, Result
Equipment	**EquipmentID**, NameNN, TotalQuantityNN, EquipmentConditionNN
MedicalRecord	**RecordID**, *MemberID*NN, MedicalConditionNN, DiagnosisDateNN

Table 4: ER attributes (Summary)

5. Relationship Justifications

Each relationship in the schema is justified below with real-world reasoning and concrete examples from the Olympia Track system.

5.1 Member ↔ Team (M:N via TeamMember)

Considering real life scenarios each player can join multiple teams and a team can have multiple players. For example, A player like "Rohit Sharma" can be in multiple teams like "Thunder Sprinters" for athletics *and* "Hoop Warriors" for basketball. Simultaneously, each team like "IITGN FC Alpha" has multiple players (Rohan, Arjun, Vikash, Harsh). This M:N relationship is resolved through the **TeamMember** junction table with composite primary key (TeamID, MemberID). Additional attributes like JoinDate and Position are stored here.

5.2 Coach → Team (1:M)

Each team must have exactly one head coach responsible for training and strategy. However, a coach like Deepak Verma (MemberID=15) can manage multiple teams — both "IITGN FC Alpha" and "IITGN FC Beta." This 1:M relationship is implemented via *CoachID* as a foreign key in the Team table, with ON DELETE RESTRICT to prevent deleting a coach with active teams.

5.3 Member ↔ Event (M:N via Participation)

Multiple players participate in each sporting event, and each player typically competes in several events. For example, Aarav participates in 100m Sprint Finals (gold, 10.85s), 200m Sprint Heats (silver, 21.50s), and Basketball 3v3 (MVP, 18 pts). The **Participation** junction table stores event-specific data including Score, Rank, and Remarks.

5.4 Event → Venue (M:1)

Each event is held at exactly one venue, but a venue can host many events throughout the year. For instance, the "Main Athletic Track" hosts the 100m Sprint Finals, 200m Sprint Heats, 400m Relay Championship, and Annual Sports Day. VenueID in Event is a foreign key with ON DELETE RESTRICT.

5.5 Member → MedicalRecord (1:M, Composition)

A player can accumulate multiple medical records over their career. Aarav has a "Hamstring Strain" (January, recovered) and a "Mild Concussion" (March, recovered). Rohan has an "Ankle Sprain" and chronic "Exercise-induced Asthma." This is a **composition** relationship — medical records have no meaning without their associated member, so ON DELETE CASCADE ensures cleanup.

5.6 Member ↔ Equipment (M:N via EquipmentIssue)

Multiple members can borrow the same type of equipment, and each member can borrow different items across sports. The **EquipmentIssue** table tracks who borrowed what, when, quantity issued, and whether it has been returned (ReturnDate = NULL indicates still checked out). A CHECK constraint ensures ReturnDate $\geq$ IssueDate.

6. Additional Constraints

6.1 Referential Integrity Actions

Foreign Key Relationship	On Delete	On Update	Justification
Team.CoachID → Member	RESTRICT	CASCADE	Cannot delete coach with active teams
Team.SportID → Sport	RESTRICT	CASCADE	Cannot delete sport with existing teams
TeamMember → Team/Member	CASCADE	CASCADE	Remove memberships on entity deletion
Event.VenueID → Venue	RESTRICT	CASCADE	Cannot delete venue with scheduled events
Event.SportID → Sport	RESTRICT	CASCADE	Cannot delete sport with existing events
Participation → Member/Event	CASCADE	CASCADE	Clean up participation records
EquipmentIssue → Equip/Member	CASCADE	CASCADE	Clean up issue tracking records
PracticeSession.TeamID → Team	CASCADE	CASCADE	Sessions meaningless without team
PracticeSession.VenueID → Venue	RESTRICT	CASCADE	Cannot delete venue with sessions
PerformanceLog → Member/Sport	CASCADE	CASCADE	Clean up performance logs
MedicalRecord → Member	CASCADE	CASCADE	Records meaningless without member

Table 5: Referential integrity actions for foreign key constraints

6.2 Logical / CHECK Constraints

Table	Constraint	Business Rule
Event	`CHECK (EndTime ≥ StartTime)`	An event cannot end before it starts
PracticeSession	`CHECK (EndTime ≥ StartTime)`	A practice session cannot end before it starts
EquipmentIssue	`CHECK (ReturnDate IS NULL OR ReturnDate ≥ IssueDate)`	Equipment cannot be returned before it was borrowed
MedicalRecord	`CHECK (RecoveryDate IS NULL OR RecoveryDate ≥ DiagnosisDate)`	Recovery cannot precede diagnosis
Member	`CHECK (Age ≥ 0)`	Age must be a positive integer
Equipment	`CHECK (TotalQuantity ≥ 0)`	Inventory cannot be negative
Participation	`CHECK (Rank ≥ 1)`	Rank must be a positive integer
Venue	`CHECK (Capacity ≥ 0)`	Venue must accommodate at least one person
Sport	`CHECK (MaxPlayersPerTeam ≥ 0)`	Team size must be positive (when applicable)
EquipmentIssue	`CHECK (Quantity ≥ 0)`	Must issue at least one unit

Table 6: Logical CHECK constraints enforcing business rules

7. Team Member Contributions

Name	Roll Number	Contribution
[Tanish Yelgoe]	[23110328]	Database schema design, SQL implementation, sample data generation, testing referential integrity
[Srivaths P]	[23110321]	UML class diagrams, relationship analysis, documentation and report writing
[Venkatakrishnan E]	[23110357]	ER diagram creation, UML-to-ER conversion, constraint specification, relationship justifications
[Anurag Singh]	[23110035]	ER diagram creation, UML-to-ER conversion, constraint specification, relationship justifications
[Arjun B Dikshit]	[23110040]	ER diagram creation, UML-to-ER conversion, constraint specification, relationship justifications

Table 7: Team member contributions)